UNIVERSITY OF LIMERICK

MSC IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CS4096 - ARTIFICIAL INTELLIGENCE FOR GAMES
Final project 2025-26

Module Leader: Prof. Gavin Wade

Project Title: Squash 'Em All

Student ID: 25020749

Student Name: Govind Santosh Pathak

Date of Submission: 28 November, 2025

Word Count: 2218

Abstract:

This report presents *Squash 'Em All*, a 2D top-down Unity game developed to demonstrate the application of Artificial Intelligence (AI) techniques in interactive environments. The artefact features an AI-controlled car tasked with pursuing zombies, while the zombies themselves exhibit reactive behaviours such as wandering, fleeing, hiding, and ultimately being destroyed when caught. The project integrates core AI methods including finite state machines (FSM) for zombie behaviour, raycasting for car obstacle avoidance, and predictive targeting for pursuit. These techniques were implemented in modular scripts to ensure clarity, reproducibility, and adaptability across different scenarios.

The game environment includes obstacles that challenge both agents to perceive and adapt dynamically, with visual feedback provided through gizmos and particle effects. Audio integration was attempted to enhance multimodal feedback, with a continuous zombie groan successfully implemented, though other state-based sounds remain incomplete. The artefact highlights both the potential and limitations of combining AI decision-making with perceptual systems in Unity, offering insights into debugging, modular design, and the challenges of synchronising audio with agent states.

By situating *Squash 'Em All* within the broader context of game AI research and practice, this report critically analyses the design choices, reflects on implementation challenges, and identifies opportunities for future improvement. The project demonstrates how relatively simple AI techniques can produce emergent and convincing gameplay behaviours, validating their relevance for both academic study and practical application.

INDEX

INTRODUCTION:

Artificial Intelligence (AI) has become a cornerstone of modern game development, enabling designers to create agents that behave in ways that feel responsive, believable, and engaging to players. The artefact developed for this project, *Squash 'Em All*, is a 2D top-down Unity game that demonstrates the application of core AI techniques in a controlled environment. The game features an AI-controlled car tasked with pursuing zombies, while the zombies themselves exhibit reactive behaviours such as wandering, fleeing, hiding, and ultimately being destroyed when caught. The environment is populated with obstacles, requiring both agents to perceive and adapt to their surroundings in real time.

The motivation behind *Squash 'Em All* is to showcase how relatively simple AI techniques, when combined thoughtfully, can produce emergent and convincing gameplay. The AI car uses raycasting to detect obstacles and implements predictive targeting to anticipate zombie movement, while zombies operate under a finite state machine (FSM) that governs their transitions between wandering, fleeing, hiding, and death states. This mirrors design patterns found in commercial games such as *Left 4 Dead*, where zombies react dynamically to player presence, or *Grand Theft Auto*, where vehicles navigate complex urban environments. By situating the artefact within this broader context, the project validates its relevance as a demonstration of game AI principles.

The artefact also integrates multimodal feedback mechanisms to enhance clarity and immersion. Visual cues such as gizmos and blood effects communicate agent decisions and state changes, while audio integration was attempted to tie sounds to AI states. Although only a continuous zombie groan was successfully implemented, this highlights both the potential and the challenges of synchronising audio with AI behaviours. Overall, *Squash 'Em All* provides a focused platform to explore the design, implementation, and evaluation of AI techniques in games, forming the basis for deeper analysis and reflection in the subsequent sections of this report.

ANALYSIS

The design of *Squash 'Em All* was informed by both theoretical considerations in game AI and practical constraints of implementation within Unity. The artefact integrates three primary techniques: finite state machines (FSMs) for zombie behaviour, raycasting for car perception and obstacle avoidance, and predictive targeting for pursuit. Each of these methods was selected on the basis of their established relevance in the literature and their suitability for demonstrating agent responsiveness in a 2D environment.

Finite State Machine (FSM)

Finite state machines remain one of the most widely adopted approaches to modelling agent behaviour in games (Millington & Funge, 2009). Their strength lies in the ability to represent discrete behavioural states and transitions in a manner that is both computationally efficient and conceptually transparent. In *Squash 'Em All*, zombies transition between wandering, fleeing, hiding, and death states. These transitions are triggered by environmental stimuli such as proximity to the car or collision events. The FSM implementation was achieved through conditional checks embedded in modular scripts, with each state encapsulating distinct movement logic. This design mirrors reactive behaviours observed in commercial titles such as *Left 4 Dead*, where non-player characters dynamically adjust their actions in response to player presence.

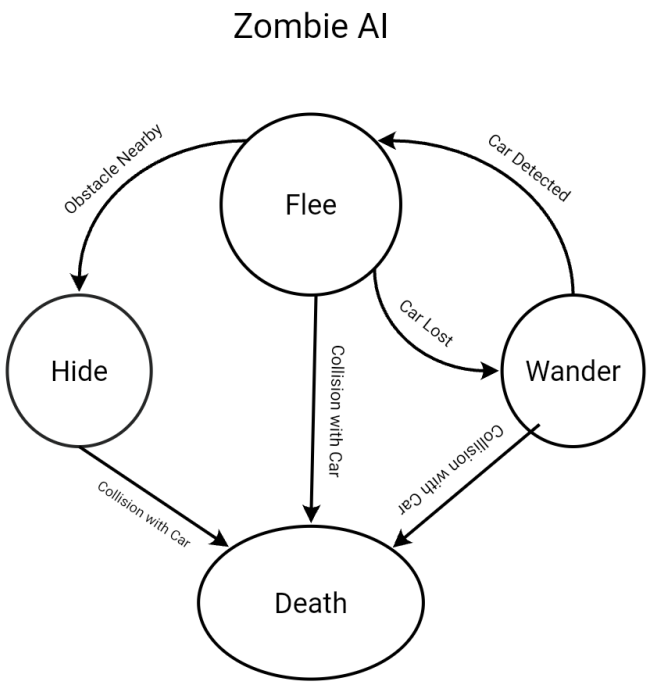


Figure 1: "Finite State Machine for Zombie AI in Squash 'Em All."

As shown in Figure 1, zombies transition between wandering, fleeing, hiding, and death states depending on environmental stimuli.

Raycasting for Car Perception and Obstacle Avoidance

The AI car relies on raycasting as its primary perceptual mechanism. Raycasting is frequently employed in both academic simulations and commercial racing games to enable agents to detect obstacles and adjust their trajectories accordingly (Rabin, 2015). In this artefact, rays are projected forward, left, and right, with LayerMasks applied to differentiate between zombies and environmental obstacles. The decision logic interprets ray collisions to determine whether the car should continue its current trajectory, steer away, or initiate a back-off routine when stuck. Debugging tools such as gizmos were employed to visualise raycasts during development, thereby facilitating iterative refinement of the avoidance logic. The inclusion of raycasting demonstrates the importance of perception in agent design and highlights the trade-off between computational simplicity and behavioural plausibility.

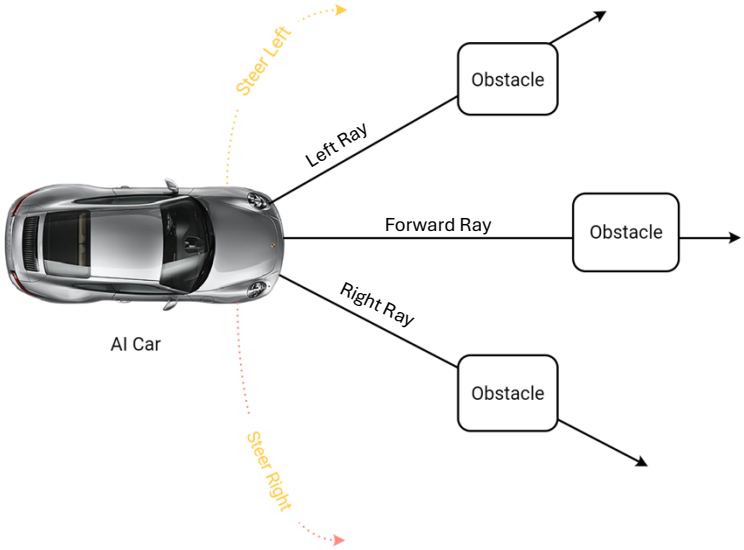


Figure 2:" Raycasting perception for AI car obstacle detection in Squash 'Em All."

As shown in Figure 2, the car projects forward, left, and right rays to detect obstacles and adjust its trajectory accordingly.

Predictive Targeting for Pursuit

Predictive targeting was incorporated into the car's pursuit behaviour to enhance realism and efficiency. Rather than steering toward the zombie's current position, the car extrapolates the zombie's velocity to estimate its future location. This anticipatory behaviour aligns with pursuit-evasion models discussed in AI literature (Isaacs & Barron, 1985) and prevents the car from trailing ineffectively behind moving targets. Implementation involved calculating the zombie's velocity vector and projecting its position forward by a short time interval, which was then used as the steering target. The result is a more convincing interception path,

demonstrating how relatively simple mathematical models can significantly improve agent performance.

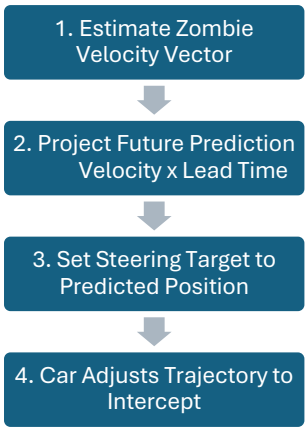


Figure 3: "Predictive targeting logic for AI car pursuit of zombies in Squash 'Em All."

As shown in Figure 3, the car estimates zombie velocity, projects a future position, and adjusts its trajectory to intercept more effectively.

Code Structure and Modularity

The implementation of Squash 'Em All was organised into modular scripts to ensure clarity and reproducibility. The ZombieAI script encapsulated finite state machine logic, while the AICarController script managed raycasting and predictive targeting for pursuit and avoidance. Collision handling and death effects were separated into the ZombieHit script, thereby maintaining single-responsibility principles. This modular approach facilitated debugging and parameter tuning through Unity's Inspector, aligning with best practices in both academic and industry contexts. The architecture demonstrates how agent behaviours can be decomposed into manageable components, supporting iterative development and transparent evaluation.

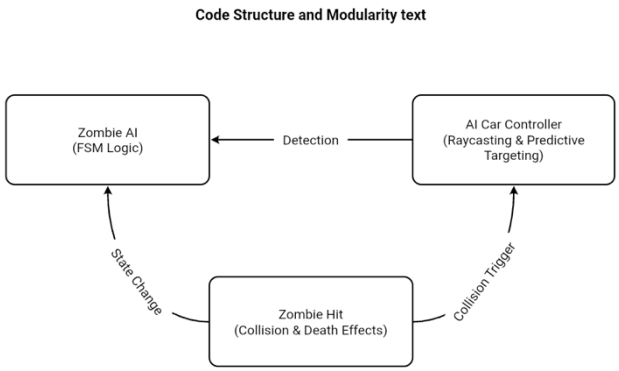


Figure 4:" Modular script architecture for Squash 'Em All."

REFLECTION

The development of *Squash 'Em All* provided valuable insights into the application of foundational AI techniques within a 2D Unity environment. The artefact successfully demonstrated finite state machines, raycasting, and predictive targeting, each of which contributed to agent responsiveness and gameplay dynamics. At the same time, the process revealed several challenges that required iterative refinement, underscoring the importance of balancing theoretical design with practical implementation.

1.Effectiveness of FSM

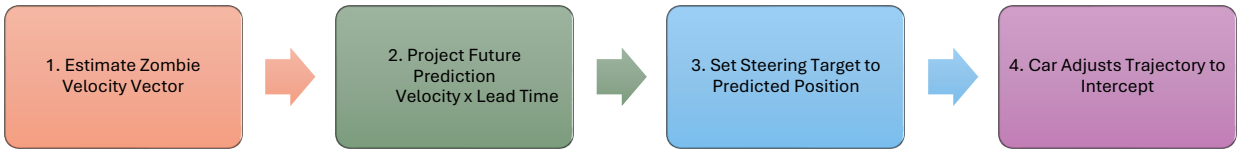
The finite state machine (FSM) governing zombie behaviour proved effective in structuring agent actions. By alternating between wandering, fleeing, hiding, and death states, zombies exhibited reactive behaviours that enhanced realism. However, testing revealed occasional instability when zombies oscillated rapidly between states near detection thresholds. This issue was mitigated by introducing cooldown timers and smoothing transitions, which reduced jitter and improved behavioural stability. The FSM therefore demonstrated its utility as a structured framework, while also highlighting the need for careful calibration of transition conditions.

2.Car Obstacle Avoidance

Raycasting formed the basis of the car's perception system, enabling obstacle detection and avoidance. Initial implementations resulted in jittering behaviour when the car encountered complex obstacle arrangements, sometimes leading to the agent becoming stuck. To address this, additional rays were introduced at slight angles, and a back-off routine was implemented when the car's velocity fell below a threshold for a sustained period. These modifications significantly reduced instances of the car becoming trapped, though occasional jitter remained in highly cluttered environments. This outcome illustrates both the strengths and limitations of raycasting: while computationally efficient, it struggles with nuanced spatial reasoning compared to more advanced pathfinding algorithms.

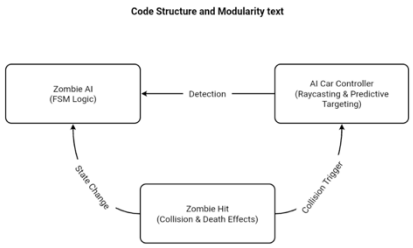
3.Pursuit and Predictive Targeting

Predictive targeting enhanced the car's pursuit behaviour by enabling interception rather than simple trailing. This produced more convincing captures and reduced the likelihood of zombies escaping indefinitely. Nevertheless, predictive targeting occasionally led to overshooting when zombies changed direction abruptly. To mitigate this, the lead time parameter was clamped and damped, ensuring smoother pursuit paths. The technique therefore demonstrated the value of anticipatory decision-making, while also underscoring the need for adaptive tuning to balance responsiveness with stability.



4. Code Modularity and Debugging

Beyond these core techniques, the project highlighted the importance of modular code structure. By separating responsibilities across scripts — with ZombieAI handling FSM logic, AICarController managing raycasting and pursuit, and ZombieHit2D controlling collision and death effects — the design adhered to single-responsibility principles. This modularity facilitated debugging, parameter tuning, and reproducibility, aligning with best practices in both academic and industry contexts. The architecture demonstrates how agent behaviours can be decomposed into manageable components, supporting iterative development and transparent evaluation.



5. Limitations and Future Work

The limitations of the artefact primarily stemmed from the simplicity of the chosen techniques. FSMs, raycasting, and predictive targeting are effective for small-scale demonstrations but lack the sophistication required for complex environments. For instance, zombies did not exhibit group behaviours or adaptive learning, and the car's avoidance logic remained reactive rather than strategic. Future work could incorporate pathfinding algorithms such as A* or navigation meshes, as well as machine learning approaches to enable adaptive behaviours. Visual and audio feedback could also be expanded to provide richer multimodal cues, enhancing immersion and clarity of agent states.

In summary, the reflection highlights both the successes and shortcomings of *Squash 'Em All*. The artefact effectively demonstrated foundational AI techniques, producing agents that were responsive and engaging within a 2D Unity environment. At the same time, the challenges encountered underscore the importance of iterative refinement, parameter tuning, and consideration of more advanced techniques for future development. By critically evaluating these outcomes, the project contributes to an understanding of how simple AI methods can be leveraged for academic demonstration while also pointing toward avenues for further exploration.

CONCLUSION:

The development of *Squash 'Em All* has demonstrated the practical application of foundational AI techniques within a 2D Unity environment. Through the integration of finite state machines, raycasting, and predictive targeting, the artefact achieved agents that were both responsive and engaging, thereby fulfilling the project's objective of showcasing AI behaviour in a game context. Each technique contributed distinct strengths: FSMs provided structured behavioural modelling, raycasting enabled efficient perception and obstacle avoidance, and predictive targeting enhanced pursuit realism. Together, these methods validated their relevance for academic demonstration and highlighted their continued importance in industry practice.

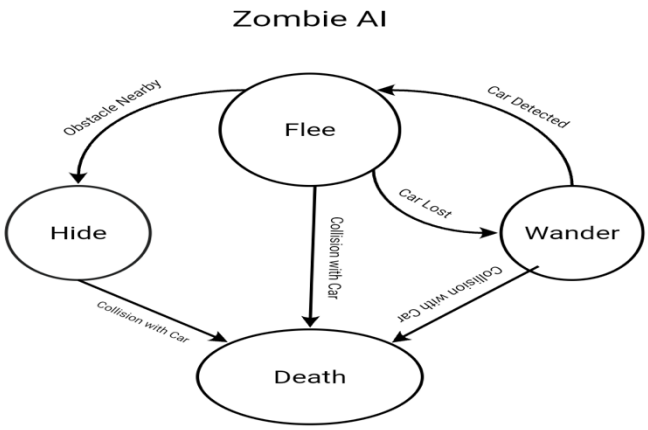
The project also revealed several challenges, including instability in state transitions, jittering in obstacle avoidance, and overshooting in pursuit behaviour. These issues were addressed through iterative refinement, parameter tuning, and modular code structuring, underscoring the importance of adaptability in AI design. While limitations remained, particularly in the simplicity of the chosen techniques, the artefact succeeded in illustrating how relatively straightforward methods can produce emergent behaviours that enrich gameplay.

From an academic perspective, the project contributes to an understanding of how AI systems can be implemented directly within Unity without reliance on external libraries. It demonstrates the pedagogical value of combining theoretical models with practical artefacts, offering insights into both the strengths and constraints of foundational approaches. Future work could extend these techniques by incorporating pathfinding algorithms, adaptive learning, or richer multimodal feedback, thereby advancing the sophistication of agent behaviours.

In conclusion, *Squash 'Em All* stands as a meaningful artefact that bridges theory and practice. It validates the utility of FSMs, raycasting, and predictive targeting as core AI methods, while also providing a platform for reflection on iterative design and future innovation. The project therefore achieves its academic aim of critically demonstrating AI agent design in a clear, reproducible, and engaging manner.

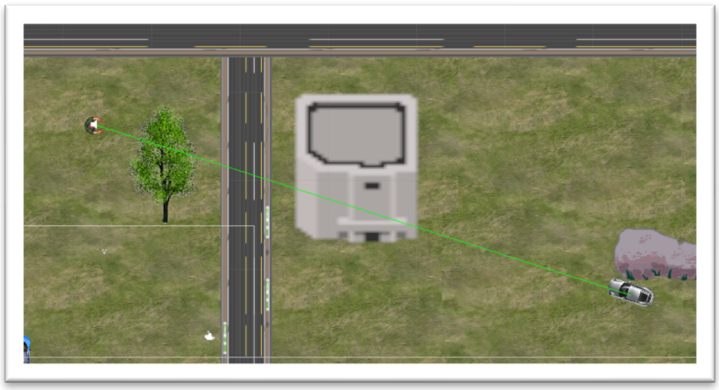
APPENDIX:

1. FSM Diagram (Zombie Behaviour)



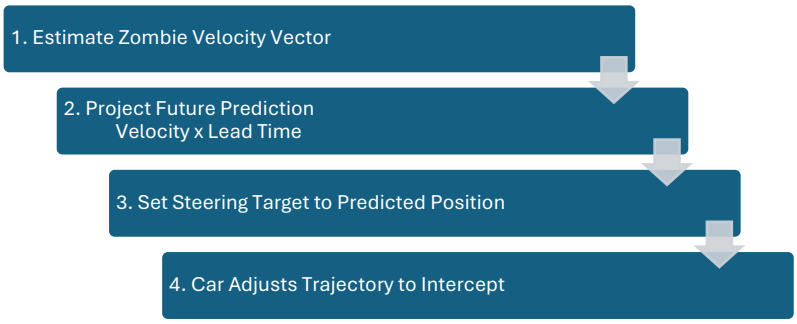
"Appendix A: Finite State Machine diagram for zombie AI."

2. Raycasting Gizmo Screenshot



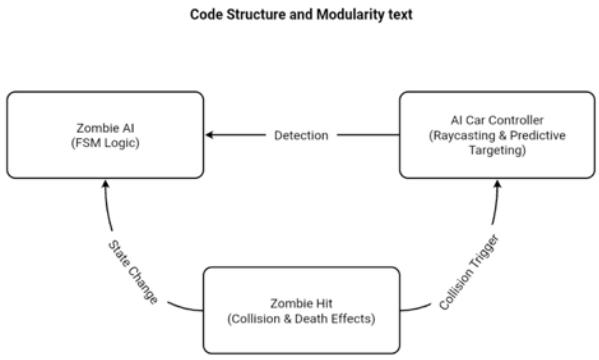
"Appendix B: Debug gizmos visualising car obstacle detection."

3. Predictive Targeting Flowchart



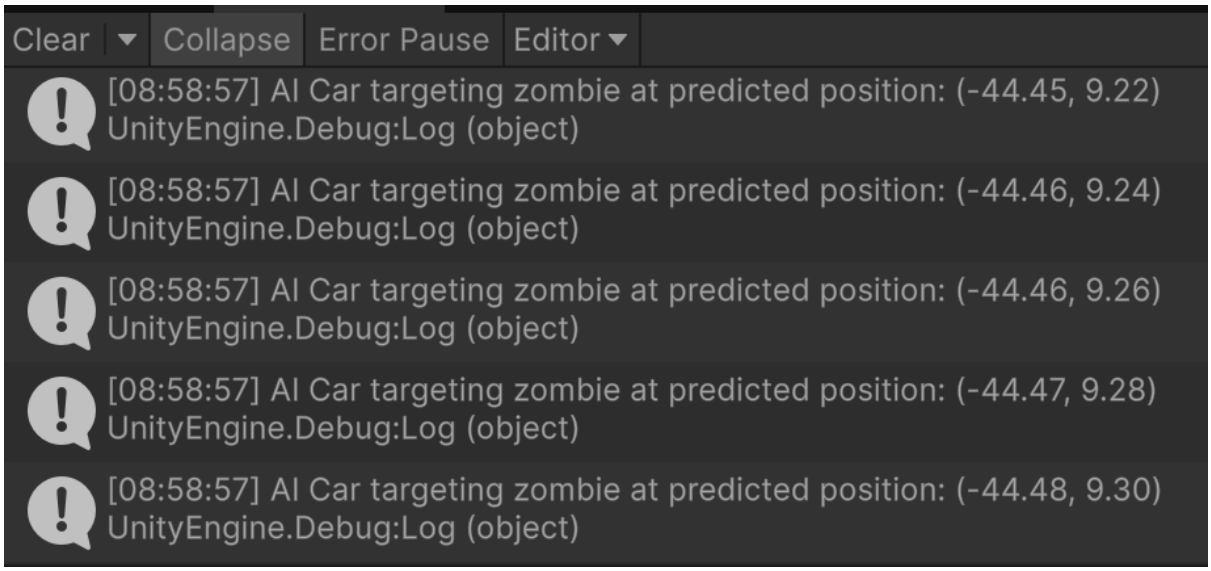
"Appendix C: Predictive targeting logic for AI car pursuit."

4. Modular Script Architecture Diagram



"Appendix D: Modular script architecture for Squash 'Em All."

5. Debug Logs (Text Snippets)



"Appendix E: Sample debug logs illustrating agent decision-making."

LINKS:

Files Link = https://drive.google.com/drive/folders/1oaP8uKYl93D5eaLnqj0f8j-XZMR-s_mi?usp=drive_link

Youtube Link = https://youtu.be/Frh88IB_e70